```
WITH monthly_kpi AS ( SELECT month,
        2
    ) AS late_15m_rate,

    COUNT(
        CASE WHEN support_contact = 'Yes' THEN 1 END
    ) AS support_contacts,

    ROUND(
        AVG(CASE WHEN support_contact = 'Yes' THEN 1 ELSE 0 END) * 100,
        2
    ) AS support_contact_rate,

    AVG(courier_rating) AS average_courier_rating,

    AVG(restaurant_rating) AS average_restaurant_rating,

    AVG(preparation_time_min) AS average_preparation_time_min,

    AVG(assignment_time_min) AS average_assignment_time_min

FROM orders
GROUP BY month
ORDER BY CASE month
    WHEN 'April' THEN 1
    WHEN 'May' THEN 2
    WHEN 'June' THEN 3
    WHEN 'July' THEN 4
END
)
SELECT month,
total_orders,
LAG(total_orders) OVER (
    ORDER BY CASE month
        WHEN 'April' THEN 1
        WHEN 'May' THEN 2
        WHEN 'June' THEN 3
        WHEN 'July' THEN 4
    END
) AS previous_total_orders,

total_orders - LAG(total_orders) OVER (
    ORDER BY CASE month
        WHEN 'April' THEN 1
        WHEN 'May' THEN 2
        WHEN 'June' THEN 3
        WHEN 'July' THEN 4
    END
) AS total_orders_change,

completed_orders,
LAG(completed_orders) OVER (
    ORDER BY CASE month
        WHEN 'April' THEN 1
        WHEN 'May' THEN 2
        WHEN 'June' THEN 3
        WHEN 'July' THEN 4
    END
) AS previous_completed_orders,

completed_orders - LAG(completed_orders) OVER (
    ORDER BY CASE month
        WHEN 'April' THEN 1
        WHEN 'May' THEN 2
        WHEN 'June' THEN 3
        WHEN 'July' THEN 4
    END
) AS completed_orders_change,

customer_cancellation_rate,
LAG(customer_cancellation_rate) OVER (
    ORDER BY CASE month
        WHEN 'April' THEN 1
        WHEN 'May' THEN 2
        WHEN 'June' THEN 3
        WHEN 'July' THEN 4
    END
) AS previous_customer_cancellation_rate,

customer_cancellation_rate - LAG(customer_cancellation_rate) OVER (
    ORDER BY CASE month
        WHEN 'April' THEN 1
        WHEN 'May' THEN 2
        WHEN 'June' THEN 3
        WHEN 'June' THEN 3
        WHEN 'July' THEN 4
    END
) AS restaurant_cancellation_rate_change,

average_order_amount_eur,
LAG(average_order_amount_eur) OVER (
    ORDER BY CASE month
        WHEN 'April' THEN 1
        WHEN 'May' THEN 2
        WHEN 'June' THEN 3
        WHEN 'July' THEN 4
    END
) AS previous_average_order_amount_eur,

average_order_amount_eur - LAG(average_order_amount_eur) OVER (
    ORDER BY CASE month
        WHEN 'April' THEN 1
        WHEN 'May' THEN 2
        WHEN 'June' THEN 3
        WHEN 'July' THEN 4
    END
) AS average_order_amount_eur_change,

LAG(average_delivery_time_min) OVER (
    ORDER BY CASE month
        WHEN 'April' THEN 1
        WHEN 'May' THEN 2
        WHEN 'June' THEN 3
        WHEN 'July' THEN 4
    END
) AS previous_average_delivery_time_min,

average_delivery_time_min - LAG(average_delivery_time_min) OVER (
    ORDER BY CASE month WHEN 'April' THEN 1
        WHEN 'May' THEN 2
        WHEN 'June' THEN 3
        WHEN 'July' THEN 4
    END
) AS average_delivery_time_min_change,

late_15m_rate,
LAG(late_15m_rate) OVER (
    ORDER BY CASE month
        WHEN 'April' THEN 1
        WHEN 'May' THEN 2
        WHEN 'June' THEN 3
        WHEN 'July' THEN 4
    END
) AS previous_late_15m_rate,

late_15m_rate - LAG(late_15m_rate) OVER (
    ORDER BY CASE month
        WHEN 'April' THEN 1
        WHEN 'May' THEN 2
        WHEN 'June' THEN 3
        WHEN 'July' THEN 4
    END
) AS late_15m_rate_change,

support_contacts,
LAG(support_contacts) OVER (
    ORDER BY CASE month
        WHEN 'April' THEN 1
        WHEN 'May' THEN 2
        WHEN 'June' THEN 3
        WHEN 'July' THEN 4
    END
) AS previous_support_contacts,
        WHEN 'June' THEN 3
        WHEN 'July' THEN 4
    END
) AS previous_support_contact_rate,

support_contact_rate - LAG(support_contact_rate) OVER (
    ORDER BY CASE month
        WHEN 'April' THEN 1
        WHEN 'May' THEN 2
        WHEN 'June' THEN 3
        WHEN 'July' THEN 4
    END
) AS support_contact_rate_change,

average_courier_rating,
LAG(average_courier_rating) OVER (
    ORDER BY CASE month
        WHEN 'April' THEN 1
        WHEN 'May' THEN 2
        WHEN 'June' THEN 3
        WHEN 'July' THEN 4
    END
) AS previous_average_courier_rating,

average_courier_rating - LAG(average_courier_rating) OVER (
    ORDER BY CASE month
        WHEN 'April' THEN 1
        WHEN 'May' THEN 2
        WHEN 'June' THEN 3
        WHEN 'July' THEN 4
    END
) AS average_courier_rating_change,

average_restaurant_rating,
LAG(average_restaurant_rating) OVER (
    ORDER BY CASE month
        WHEN 'April' THEN 1
        WHEN 'May' THEN 2
        WHEN 'June' THEN 3
        WHEN 'July' THEN 4
    END
) AS previous_average_restaurant_rating,

average_restaurant_rating - LAG(average_restaurant_rating) OVER (
    ORDER BY CASE month
        WHEN 'April' THEN 1
        WHEN 'May' THEN 2
        WHEN 'June' THEN 3
        WHEN 'July' THEN 4
    END
) AS average_restaurant_rating_change,

average_preparation_time_min,
LAG(average_preparation_time_min) OVER (
    ORDER BY CASE month
        WHEN 'April' THEN 1
        WHEN 'May' THEN 2
        WHEN 'June' THEN 3
        WHEN 'July' THEN 4
    END
) AS previous_average_preparation_time_min,

average_preparation_time_min - LAG(average_preparation_time_min) OVER (
    ORDER BY CASE month
        WHEN 'April' THEN 1
        WHEN 'May' THEN 2
        WHEN 'June' THEN 3
        WHEN 'July' THEN 4
    END
) AS average_preparation_time_min_change,

average_assignment_time_min,
LAG(average_assignment_time_min) OVER (
    ORDER BY CASE month
        WHEN 'April' THEN 1
        WHEN 'May' THEN 2
        WHEN 'June' THEN 3
        WHEN 'July' THEN 4
    END
) AS previous_average_assignment_time_min,

average_assignment_time_min - LAG(average_assignment_time_min) OVER (
    ORDER BY CASE month
        WHEN 'April' THEN 1
        WHEN 'May' THEN 2
        WHEN 'June' THEN 3
        WHEN 'July' THEN 4
    END
WITH delivery_bottleneck AS ( SELECT month,
        AVG(
            CASE
                WHEN late_15m = 'Yes' THEN 1
                ELSE 0
            END
        ) * 100,
        2
    ) AS late_15m_rate

FROM orders

GROUP BY month
),
bottleneck_changes AS ( SELECT month,
    average_preparation_time_min,

    LAG(average_preparation_time_min) OVER (
        ORDER BY CASE month
            WHEN 'April' THEN 1
            WHEN 'May' THEN 2
            WHEN 'June' THEN 3
            WHEN 'July' THEN 4
        END
    ) AS previous_preparation_time_min,

    average_preparation_time_min
    -
    LAG(average_preparation_time_min) OVER (
        ORDER BY CASE month
            WHEN 'April' THEN 1
            WHEN 'May' THEN 2
            WHEN 'June' THEN 3
            WHEN 'July' THEN 4
        END
    ) AS preparation_time_change,

    average_assignment_time_min,

    LAG(average_assignment_time_min) OVER (
        ORDER BY CASE month
            WHEN 'April' THEN 1
            WHEN 'May' THEN 2
            WHEN 'June' THEN 3
            WHEN 'July' THEN 4
        END
    ) AS previous_assignment_time_min,

    average_assignment_time_min
    -
    LAG(average_assignment_time_min) OVER (
        ORDER BY CASE month
            WHEN 'April' THEN 1
            WHEN 'May' THEN 2
            WHEN 'June' THEN 3
            WHEN 'July' THEN 4
        END
    ) AS assignment_time_change,

    average_delivery_time_min,

    LAG(average_delivery_time_min) OVER (
        ORDER BY CASE month
            WHEN 'April' THEN 1
            WHEN 'May' THEN 2
            WHEN 'June' THEN 3
            WHEN 'July' THEN 4
        END
    ) AS previous_delivery_time_min,

    average_delivery_time_min
    -
    LAG(average_delivery_time_min) OVER (
        ORDER BY CASE month
            WHEN 'April' THEN 1
            WHEN 'May' THEN 2
            WHEN 'June' THEN 3
            WHEN 'July' THEN 4
        END
    ) AS delivery_time_change,

    late_15m_rate,

    LAG(late_15m_rate) OVER (
        ORDER BY CASE month
            WHEN 'April' THEN 1
            WHEN 'May' THEN 2
            WHEN 'June' THEN 3
            WHEN 'July' THEN 4
        END
    ) AS previous_late_15m_rate,

    late_15m_rate
    -
    LAG(late_15m_rate) OVER (
        ORDER BY CASE month
            WHEN 'April' THEN 1
            WHEN 'May' THEN 2
            WHEN 'June' THEN 3
            WHEN 'July' THEN 4
        END
    ) AS late_15m_rate_change

FROM delivery_bottleneck
)
SELECT month,
average_preparation_time_min,
previous_preparation_time_min,
preparation_time_change,

average_assignment_time_min,
previous_assignment_time_min,
assignment_time_change,

average_delivery_time_min,
previous_delivery_time_min,
delivery_time_change,

late_15m_rate,
previous_late_15m_rate,
late_15m_rate_change
FROM bottleneck_changes
ORDER BY average_delivery_time_min DESC;

-- 4. Delay & Customer Behaviour
WITH late_15m_statistics AS ( SELECT late_15m,
    COUNT(order_id) AS total_orders,

    COUNT(
        CASE
            WHEN customer_cancelled = 'Yes' THEN 1
        END
    ) AS customer_cancellations,

    ROUND(
        AVG(
            CASE
                WHEN customer_cancelled = 'Yes' THEN 1
                ELSE 0
            END
        ) * 100,
        2
    ) AS customer_cancellation_rate,

    COUNT(
        CASE
            WHEN support_contact = 'Yes' THEN 1
        END
    ) AS support_contacts,

    ROUND(
        AVG(
            CASE
                WHEN support_contact = 'Yes' THEN 1
                ELSE 0
            END
        ) * 100,
        2
    ) AS support_contact_rate,

    AVG(delivery_time_min)
        AS average_delivery_time_min,

    AVG(courier_rating)
        AS average_courier_rating,

    AVG(restaurant_rating)
        AS average_restaurant_rating

FROM orders

WHERE late_15m IN ('Yes', 'No')

GROUP BY late_15m
),
late_orders AS ( SELECT total_orders, customer_cancellations, customer_cancellation_rate, support_contacts, support_conta
FROM late_15m_statistics

WHERE late_15m = 'Yes'
),
non_late_orders AS ( SELECT total_orders, customer_cancellations, customer_cancellation_rate, support_contacts, support_c
FROM late_15m_statistics

WHERE late_15m = 'No'
)
SELECT late_orders.total_orders AS late_orders,
non_late_orders.total_orders
    AS non_late_orders,

late_orders.customer_cancellations
    AS late_customer_cancellations,

non_late_orders.customer_cancellations
    AS non_late_customer_cancellations,

late_orders.customer_cancellation_rate
    AS late_customer_cancellation_rate,

non_late_orders.customer_cancellation_rate
    AS non_late_customer_cancellation_rate,

late_orders.customer_cancellation_rate
    - non_late_orders.customer_cancellation_rate
    AS customer_cancellation_rate_difference,

late_orders.support_contacts
    AS late_support_contacts,

non_late_orders.support_contacts
    AS non_late_support_contacts,

late_orders.support_contact_rate
    AS late_support_contact_rate,

non_late_orders.support_contact_rate
    AS non_late_support_contact_rate,

late_orders.support_contact_rate
    - non_late_orders.support_contact_rate
    AS support_contact_rate_difference,

late_orders.average_delivery_time_min
    AS late_average_delivery_time_min,

non_late_orders.average_delivery_time_min
    AS non_late_average_delivery_time_min,

late_orders.average_delivery_time_min
    - non_late_orders.average_delivery_time_min
    AS average_delivery_time_difference,

late_orders.average_courier_rating
    AS late_average_courier_rating,

non_late_orders.average_courier_rating
    AS non_late_average_courier_rating,

late_orders.average_courier_rating
    - non_late_orders.average_courier_rating
    AS courier_rating_difference,

late_orders.average_restaurant_rating
    AS late_average_restaurant_rating,

non_late_orders.average_restaurant_rating
    AS non_late_average_restaurant_rating,

late_orders.average_restaurant_rating
    - non_late_orders.average_restaurant_rating
    AS restaurant_rating_difference
FROM late_orders CROSS JOIN non_late_orders;

-- Control -
SELECT
    customer_cancelled,
    late_15m,
    COUNT(*) AS orders
FROM orders
GROUP BY customer_cancelled, late_15m
ORDER BY customer_cancelled, late_15m;

-- Query revised after data validation identified a logical inconsistency
-- (Even my confusion is confused)

WITH late_15m_statistics AS ( SELECT late_15m,
    COUNT(order_id) AS total_orders,

    COUNT(
        CASE
            WHEN support_contact = 'Yes' THEN 1
        END
    ) AS support_contacts,

    ROUND(
        AVG(
            CASE
                WHEN support_contact = 'Yes' THEN 1
                ELSE 0
            END
        ) * 100,
        2
    ) AS support_contact_rate,

    AVG(delivery_time_min)
        AS average_delivery_time_min,
        AS average_courier_rating,

    AVG(restaurant_rating)
        AS average_restaurant_rating

FROM orders

WHERE late_15m IN ('Yes', 'No')

GROUP BY late_15m
),
late_orders AS ( SELECT total_orders, support_contacts, support_contact_rate, average_delivery_time_min, average_courier_
FROM late_15m_statistics

WHERE late_15m = 'Yes'
),
non_late_orders AS ( SELECT total_orders, support_contacts, support_contact_rate, average_delivery_time_min, average_cou
FROM late_15m_statistics

WHERE late_15m = 'No'
)
SELECT late_orders.total_orders AS late_orders,
non_late_orders.total_orders
    AS non_late_orders,

late_orders.support_contacts
    AS late_support_contacts,

non_late_orders.support_contacts
    AS non_late_support_contacts,

late_orders.support_contact_rate
    AS late_support_contact_rate,

non_late_orders.support_contact_rate
    AS non_late_support_contact_rate,

late_orders.support_contact_rate
    - non_late_orders.support_contact_rate
    AS support_contact_rate_difference,

late_orders.average_delivery_time_min
    AS late_average_delivery_time_min,

non_late_orders.average_delivery_time_min
    AS non_late_average_delivery_time_min,

late_orders.average_delivery_time_min
    - non_late_orders.average_delivery_time_min
    AS average_delivery_time_difference,

late_orders.average_courier_rating
    AS late_average_courier_rating,

non_late_orders.average_courier_rating
    AS non_late_average_courier_rating,

late_orders.average_courier_rating
    - non_late_orders.average_courier_rating
    AS courier_rating_difference,

late_orders.average_restaurant_rating
    AS late_average_restaurant_rating,

non_late_orders.average_restaurant_rating
    AS non_late_average_restaurant_rating,

late_orders.average_restaurant_rating
    - non_late_orders.average_restaurant_rating
    AS restaurant_rating_difference
FROM late_orders CROSS JOIN non_late_orders;

-- 5. Business Impact
WITH Business_Impact_Estimate AS ( SELECT month,
    COUNT(order_id) AS total_orders,

    COUNT(
        CASE
            WHEN order_status = 'Completed' THEN 1
        END
    ) AS completed_orders,

    ROUND(
        AVG(
            CASE
                WHEN customer_cancelled = 'Yes' THEN 1
                ELSE 0
            END
        ) * 100,
        2
    ) AS customer_cancellation_rate,

    ROUND(
        AVG(
            CASE
                WHEN restaurant_cancelled = 'Yes' THEN 1
                ELSE 0
            END
        ) * 100,
        2
    ) AS restaurant_cancellation_rate,

    AVG(
        CAST(
            NULLIF(
                REPLACE(
                    TRIM(order_amount_eur),
                    '€',
                    ''
                ),
                ''
            )
            AS NUMERIC
        )
    ) AS average_order_amount_eur,

    AVG(delivery_time_min)
        AS average_delivery_time_min,

    ROUND(
        AVG(
            CASE
                WHEN late_15m = 'Yes' THEN 1
                ELSE 0
            END
        ) * 100,
        2
    ) AS late_15m_rate,

    COUNT(
        CASE
            WHEN support_contact = 'Yes' THEN 1
        END
    ) AS support_contacts,

    ROUND(
        AVG(
            CASE
                WHEN support_contact = 'Yes' THEN 1
                ELSE 0
            END
        ) * 100,
        2
    ) AS support_contact_rate,

    AVG(courier_rating)
        AS average_courier_rating,

    AVG(restaurant_rating)
        AS average_restaurant_rating,

    AVG(preparation_time_min)
        AS average_preparation_time_min,

    AVG(assignment_time_min)
        AS average_assignment_time_min

FROM orders

GROUP BY month
),
july_total_orders AS ( SELECT MAX( CASE WHEN month = 'July' THEN total_orders END ) AS july_total_orders
FROM Business_Impact_Estimate
),
july_cancellation_rate AS ( SELECT MAX( CASE WHEN month = 'July' THEN customer_cancellation_rate END ) AS july_cancellat
FROM Business_Impact_Estimate
),
june_cancellation_rate AS ( SELECT MAX( CASE WHEN month = 'June' THEN customer_cancellation_rate END ) AS june_cancellat
FROM Business_Impact_Estimate
),
july_average_order_amount AS ( SELECT MAX( CASE WHEN month = 'July' THEN average_order_amount_eur END ) AS july_average_
FROM Business_Impact_Estimate
),
actual_july_cancellations AS ( SELECT j.july_total_orders, r.july_cancellation_rate,
    (
        j.july_total_orders
        * r.july_cancellation_rate
        / 100
    ) AS actual_july_cancellations

FROM july_total_orders j

CROSS JOIN july_cancellation_rate r
),
expected_july_cancellations AS ( SELECT j.july_total_orders, m.june_cancellation_rate,
    (
        j.july_total_orders
        * m.june_cancellation_rate
        / 100
    ) AS expected_july_cancellations

FROM july_total_orders j

CROSS JOIN june_cancellation_rate m
),
cancellation_difference AS ( SELECT a.july_total_orders, a.july_cancellation_rate, m.june_cancellation_rate, a.actual_ju
    (
        a.actual_july_cancellations
        - e.expected_july_cancellations
    ) AS potentially_retained_orders

FROM actual_july_cancellations a

CROSS JOIN expected_july_cancellations e

CROSS JOIN june_cancellation_rate m
),
potential_gmv AS ( SELECT c.july_total_orders, c.july_cancellation_rate, c.june_cancellation_rate, c.actual_july_cancell
    (
        c.potentially_retained_orders
        * d.july_average_order_amount
    ) AS potential_additional_gmv

FROM cancellation_difference c

CROSS JOIN july_average_order_amount d
)
SELECT july_total_orders, july_cancellation_rate, june_cancellation_rate, actual_july_cancellations, expected_july_cance
FROM potential_gmv;
```